BrainAI

ProSuite Industrial

PROSUITE INDUSTRIAL

PRODUCT DOCUMENTATION

BrainAI (Inhaber: Sascha Alexander Köhne)

BrainAI Inhaber: Sascha Alexander Köhne	PROSUITE INDUSTRIAL	🟢 PUBLIC CLASSIFICATION
Core Architecture & Development	**Technical Documentation**	Access Level: Public / Beta

---

Revision History

Version	Date	Author	Status	Description
4.5.0	2026-02-28	S. Köhne	Stable	Initial Release: Modular JS Architecture (Zero-Trust)

1. Introduction: ProSuite JS Architecture

The JavaScript ecosystem of the ProSuite (ProWB Edition) has been entirely re-architected from the ground up, strictly adhering to the **Separation of Concerns (SoC)** principle and uncompromising **Zero-Trust premises**.

Unlike conventional web frameworks (such as React or Angular)—where UI, routing, and business logic are heavily intertwined—the ProSuite frontend is divided into **7 highly specialized, mutually isolated modules**. This architectural strictness guarantees that memory leaks, Cross-Site Scripting (XSS) attacks, or DOM-related crashes are systematically prevented, ensuring the underlying cryptography remains absolutely invulnerable.

To guarantee flawless initialization and dependency resolution, the ProWB (C-Builder) injects these files into the `index.html` monolith in strict ascending numerical order (`00_` through `60_`).

---

2. Module Specifications ("Where is what?")

`00_bridge.js` (Host Interface)

Responsibility: Bidirectional communication with potential host environments.

- Acts as a decoupled interfacing layer between the web frontend and native execution containers (WASM-Host, MAUI, WebView2).
- Utilizes `CustomEvent` dispatching (`bridge-data`) to distribute incoming host payloads across the browser environment without forcing hard-coded script dependencies.

`10_base_app.js` (Boot & Core Loader)

Responsibility: Global state management and system initialization.

- Defines the primary global namespace `app` and `app.state` (which securely holds the `activeKey` temporarily in RAM).
- Parses the globally injected C-constant `PRO_WASM_CORE` (provided by ProWB) to dynamically load the required WebAssembly core (`prokey.js`, `proed.js`,

etc.).
- Broadcasts the critical `app-ready` event once the DOM and basic structures are fully initialized.

`20_navigation.js` (Routing & History)

Responsibility: URL handling and View switching.

- Encapsulates the core `app.navigate(viewId)` routine.
- Toggles the visibility of the `<section>` blocks dynamically generated by the ProWB (UI views and parsed Markdown manuals).
- **Security & UX Feature:** Integrates a forced scroll-fix, ensuring that when a lengthy manual (`doc_...`) is accessed, the viewport instantly resets to the top.
- Manages browser history (`pushState`) to ensure native "Back/Forward" button compatibility without page reloads.

`30_ui_general.js` (Theme & UI Management)

Responsibility: Purely visual logic with zero cryptographic relevance.

- Houses the logic for dynamic theme switching (`light`, `matrix`, `proedc`) via `document.body.setAttribute`.
- Manages generic tab-switching (e.g., toggling between text and file input modalities).
- Handles the intelligent auto-swap UX logic for input/output fields when users switch between "Encrypt" and "Decrypt" modes.

`40_crypto_core.js` (WASM Bindings & Cryptography)

Responsibility: The exclusive bridge to the native C-WASM functions.

- **Heap Management:** Exposes `getMem()` for direct byte-level access to the `HEAPU8` (WASM operational memory).
- **Hex-Conversions:** Contains sanitized `toHex` and `fromHex` parsing logic.
- **WASM Operations:** Hosts the execution layers for `handleNewSession` (ProKey), `computeProHash` (ProHash), and `executeProED` (Text & File Envelope Encryption).

- **Zero-Trust Enforcement:** Every operational call is strictly wrapped in `try/catch/finally` blocks. The `finally` block mandates a call to `ProSecurity.secureFree()`, guaranteeing that pointers and sensitive data traces are physically wiped from RAM, even if the C-engine throws a fatal exception.

`50_terminal.js` (Logging & Ring-Buffer)

Responsibility: Visual audit feedback and DOM protection.

- Encapsulates all console outputs mirroring into the ProSuite HTML terminal.
- **XSS-Protection:** Every injected log entry is aggressively sanitized via the internal `_escapeHtml` function.
- **DOM-Protection (Ring-Buffer):** Hard-limits the terminal display to a maximum of 100 entries (`MAX_LINES`). This physically prevents the browser from freezing due to DOM overload during high-speed, automated audit runs (e.g., via ProTU).

`60_security.js` (Zero-Trust Panic & Memory Wipe)

Responsibility: Physical memory hygiene and emergency reset procedures.

- **`secureFree(ptr, size)`:** The most critical building block of the suite. It physically overwrites the allocated memory block within the WASM heap with zeros (`mem.fill(0)`) *before* returning the pointer to the Emscripten garbage collector.
- **`panicWipe()`:** The "Nuclear Option". It forcefully overwrites the active session key in RAM with zeros, purges the `localStorage`, resets all DOM input fields, and forces a hard browser reboot to clear all caches.
- **`checkAutoLoad()`:** Securely validates stored keys from `localStorage` against strict byte-length constraints (32, 64, or 128 bytes) before mounting them into the operational state.

3. Event-Driven Architecture (Decoupling)

To prevent code entanglement (spaghetti code), the modules communicate primarily via native DOM events. A module does not need to know whether another module exists to function properly.

Event Name	Triggered By	Reaction / Listener
`app-ready`	`10_base_app.js` (Post-Boot)	All subsequent scripts use this to safely register their UI listeners (`onclick`, `change`).
`core-ready`	`10_base_app.js` (Post WASM-Load)	`50_terminal.js` logs the successful mounting of the C-Engine.
`security-wipe-complete`	`60_security.js` (Post-Panic)	Triggers UI resets and final state cleanup before the hard reboot.

4. Code Hygiene & Memory Vectors

- **Global Namespace:** The only permitted global object is `app`, alongside strictly defined functional constants (`ProUI`, `ProCrypto`, `ProTerminal`, and `ProSecurity`).
- **Garbage Collection Evasion:** Highly sensitive data (such as cryptographic keys or plaintext payloads) are **never** entrusted to the JavaScript Garbage Collector, as JS cannot guarantee timely physical deletion. Instead, payloads are manually pushed into the native C-Heap (`_malloc`), processed, and actively obliterated using `secureFree`.

© 2026 Sascha Köhne	✉️ **Contact for KYC:** koehne83@googlemail.com	🛡️ **ProTU-Verified**

System Architect	BrainAI Inhaber : Sascha Alexander Köhne	We don't need **BRUTEFORCE**, we know **PHYSICS**